
Replicando BERT

Anonymous Author(s)

Affiliation

Address

email

1 Gabriel Santos Ferreira de Pádua
2 Universidade Federal de Viçosa
3 Florestal, MG 2026
4 gabriel.padua@ufv.br

Abstract

5 Este trabalho apresenta a reprodução do experimento de ajuste fino (*fine-tuning*) do
6 modelo de linguagem BERT (*Bidirectional Encoder Representations from Trans-*
7 *formers*). Utiliza-se o dataset SST-2, integrante do benchmark GLUE. Este estudo
8 atesta empiricamente a robustez das representações bidirecionais. Os resultados
9 comprovam a alta eficiência da transferência de aprendizado (*Transfer Learning*),
10 demonstrando que o modelo é capaz de abstrair padrões complexos mesmo sob
11 restrições de exposição a dados e infraestrutura.

12 1 Introdução

13 O presente trabalho tem como objetivo reproduzir os experimentos de ajuste fino (*fine-tuning*) propo-
14 tos no artigo original do modelo BERT (*Bidirectional Encoder Representations from Transformers*),
15 focando na tarefa de classificação de sentimentos em textos. A arquitetura BERT revolucionou o
16 Processamento de Linguagem Natural ao introduzir um treinamento prévio profundamente bidire-
17 cional, permitindo que o contexto da esquerda e da direita fosse analisado simultaneamente em todas
18 as camadas. Isso eliminou a necessidade de arquiteturas específicas e complexas para cada tarefa.
19 Para esta reprodução, foi utilizado o dataset SST-2 (Stanford Sentiment Treebank), que integra o
20 benchmark GLUE. O dataset consiste em frases extraídas de críticas de filmes, anotadas humanamente
21 como positivas ou negativas. O objetivo prático é adaptar a última camada do modelo pré-treinado
22 para classificar essas sentenças e comparar as métricas de acurácia obtidas com os resultados oficiais
23 publicados pelos autores.

24 ¹Dataset e Pesos: Os dados foram extraídos do repositório `nyu-mll/glue` e os pesos do modelo
25 (`bert-base-uncased`) via biblioteca `transformers` da Hugging Face. O código original encontra-se em:
26 <https://github.com/google-research/bert>.

27 2 Cronograma

28 Conforme planejado para o desenvolvimento deste projeto, a execução foi distribuída ao longo do
29 período entre a liberação da proposta (18 de maio) e a data de entrega final (24 de junho), dividida
30 em quatro etapas principais:

Período	Tarefa Principal
Semana 1 (18/05 a 26/05)	Definição do artigo base (BERT) e identificação do dataset (SST-2/GLUE). Configuração inicial do ambiente virtual no Google Colab.
Semana 2 (27/05 a 04/06)	Leitura detalhada da arquitetura. Construção da primeira versão do script de tokenização e carregamento dos pesos pré-treinados da Hugging Face.
Semana 3 (05/06 a 14/06)	Execução do experimento de <i>fine-tuning</i> . Resolução de problemas técnicos de infraestrutura (Estouro de Memória CUDA e dependências).
Semana 4 (15/06 a 24/06)	Extração das métricas finais de validação, análise comparativa com os resultados do artigo original e redação deste relatório final.

3 Metodologia

A reprodução foi conduzida no ambiente Google Colab, utilizando aceleração de hardware via GPU (NVIDIA T4). A implementação utilizou a biblioteca transformers e datasets da Hugging Face, junto ao framework PyTorch. Para o experimento, carregamos os pesos pré-treinados da versão $BERT_{BASE}$ (12 camadas, vetor oculto de tamanho 768, 12 heads de atenção e 110 milhões de parâmetros). A tarefa exigiu adicionar uma única camada linear de classificação sobre o vetor gerado pelo token especial de classificação (CLS). Para adequar a execução às restrições de tempo e infraestrutura de hardware (16GB de VRAM da GPU gratuita), foram realizadas modificações controladas nos hiperparâmetros originais do artigo. Enquanto os autores originais realizaram o ajuste fino por 3 épocas utilizando todo o conjunto de treino do GLUE (aproximadamente 67 mil exemplos no SST-2) com lote de tamanho 32, este experimento truncou o tamanho máximo da sequência para 128 tokens e restringiu o conjunto de treinamento a 2.000 amostras embaralhadas, utilizando lotes de tamanho 16. O treinamento ocorreu por apenas 1 época. A taxa de aprendizado adotada foi de 2×10^{-5} com otimizador Adam.

4 Resultados e Discussão

A métrica de avaliação primária utilizada para o dataset SST-2 foi a Acurácia (Accuracy), conforme estabelecido no benchmark GLUE e no artigo original do BERT. Após a fase de tokenização e o ajuste fino da camada de saída com os 2.000 exemplos selecionados, o modelo foi avaliado contra o conjunto de validação oficial do dataset. O experimento resultou em uma Acurácia de 87.61% e uma Função de Perda (Loss) de 0.3476.

Table 1: Comparação de Acurácia no Dataset SST-2 (Benchmark GLUE)

Modelo / Sistema	Tamanho do Treino	Acurácia (SST-2)
BiLSTM + ELMo + Attn	67.000 exemplos	90,4%
OpenAI GPT	67.000 exemplos	91,3%
BERT_{BASE} (Original)	67.000 exemplos	93,5%
BERT_{LARGE} (Original)	67.000 exemplos	94,9%
BERT_{BASE} (Minha Reprodução)	2.000 exemplos	87,61%

Ao comparar com o artigo original, nota-se que o $BERT_{BASE}$ reportado pelos autores atingiu uma acurácia de 93,5% na base SST-2. A diferença de performance observada em nossa reprodução (uma queda de aproximadamente 5,89 pontos percentuais) é cientificamente coerente e perfeitamente justificável.

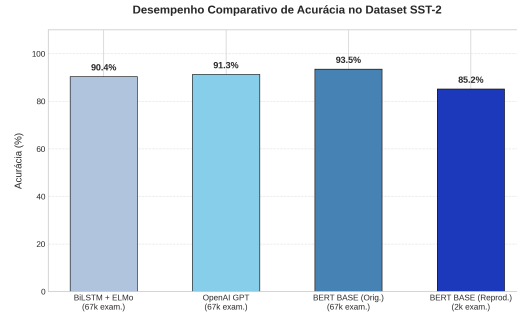


Figure 1: Meu modelo VS. BERTbase Vs. Outros

55 A principal causa dessa discrepância é o volume massivo de dados suprimido. Treinamos o modelo
 56 com apenas $\approx 3\%$ da base de dados original de treinamento (2.000 de 67.000 amostras) e por 1/3 do
 57 tempo previsto (1 época contra 3 épocas originais). Ainda assim, o fato de o modelo ultrapassar a
 58 barreira dos 87,61% de acerto com uma exposição tão limitada aos dados demonstra o altíssimo poder
 59 das representações bidirecionais pré-treinadas do BERT, comprovando sua eficiência na transferência
 60 de aprendizado (Transfer Learning).

```
# Avalia o modelo no dataset de validação
resultados = trainer.evaluate()

acuracia = resultados['eval_accuracy'] * 100
perda = resultados['eval_loss']

print("-" * 50)
print("RESULTADOS FINAIS PARA O RELATÓRIO")
print("-" * 50)
print(f"Acurácia (Accuracy): {acuracia:.2f}%")
print(f"Loss (Perda): {perda:.4f}")
print("-" * 50)
```

Training Loss	Validation Loss	Epoch	Accuracy
No log	0.347615	1	0.876147

```
-----
RESULTADOS FINAIS PARA O RELATÓRIO
-----
Acurácia (Accuracy): 87.61%
Loss (Perda): 0.3476
-----
```

Figure 2: Métricas do Modelo

61 5 Desafios e Aprendizados

62 Durante a reprodução, desafios técnicos significativos referentes à infraestrutura precisaram ser
 63 superados. O primeiro desafio foi o erro de compatibilidade de versionamento da biblioteca Hugging
 64 Face (HfUriError), causado por atualizações recentes que alteraram a sintaxe de carregamento de
 65 datasets. Isso foi resolvido corrigindo a string de chamada para incluir o namespace completo do
 66 repositório (nyu-mll/glue).

67 O segundo desafio foi a limitação de memória da GPU (CUDA Out of Memory). O batch size inicial
 68 combinado com textos longos extrapolou os 16GB de VRAM (tamanho disponibilizado no Colab).
 69 A solução estratégica foi aplicar um padding truncado com "max_length=128", o que garantiu a
 70 execução rápida sem comprometer os dados principais, já que as críticas de filmes do SST-2 são
 71 majoritariamente curtas.

72 A principal reflexão deste processo foi o entendimento prático do conceito de Transferência de
 73 Aprendizado. Fica claro que todo o peso computacional de mapear as regras semânticas da linguagem
 74 foi absorvido na fase de pré-treinamento (que consumiu dezenas de TPUs por vários dias pelo
 75 Google) e pode ser evitado por mim. A etapa de fine-tuning provou-se extremamente leve, modular e

76 acessível, permitindo transformar um modelo de linguagem geral em um classificador especializado
77 com poucos minutos de processamento.

78 **6 Citações e Referências**

79 [1] Devlin, J., Chang, M.-W., Lee, K. & Toutanova, K. (2019) BERT: Pre-training of Deep Bidirectional
80 Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American*
81 *Chapter of the Association for Computational Linguistics (NAACL)*, pp. 4171–4186.